	**Carátula para entrega de prácticas**

Laboratorio de computación salas A y B

Profesor:	René Adrián Dávila Pérez
Asignatura:	Programación Orientada a Objetos
Grupo:	7
No de Práctica(s):	9
Integrante(s):	120002850 323064039
	323071378 323214636
	426059576 426058744
Semestre:	2027-1
Fecha de entrega:	27-Octubre-2026
Obervaciones:	

CALIFICACIÓN: _____

Índice

1. Introducción **3**

1.1. Planteamiento del problema 3

1.2. Motivación 3

1.3. Objetivos 3

2. Marco Teórico **4**

2.1. Fundamentos del Lenguaje Unificado de Modelado (UML) 4

2.2. Diagramas Estructurales: Clases y Paquetes 4

2.3. Anatomía de una Clase y Convenciones Notacionales 4

2.4. Relaciones entre Clases y Cardinalidades 5

2.5. Diagramas de Comportamiento: Secuencia y Actividades 5

3. Desarrollo **6**

3.1. Modelado UML de la Práctica 5: Encapsulamiento y Control 6

3.2. Modelado UML de la Práctica 6: Organización Modular y Paquetes . 10

3.3. Modelado UML de la Práctica 7: Jerarquía Profunda de Cuadriláteros 13

3.4. Modelado UML de la Práctica 8: Interfaces y Polimorfismo 18

3.5. Metamodelo Comparativo y Visión Global de Práctica 9 22

4. Resultados **23**

4.1. Verificación de Conformidad Estructural y Sintáctica 23

4.2. Análisis de Trazabilidad entre Modelos y Código 24

5. Conclusiones **25**

1. Introducción

1.1. Planteamiento del problema

El desarrollo de sistemas de software mediante el paradigma orientado a objetos exige representaciones gráficas formales que superen la ambigüedad del código fuente aislado y permitan comunicar la estructura, jerarquías y flujos dinámicos de las soluciones implementadas. A lo largo del curso se han desarrollado arquitecturas progresivas que abarcan desde el encapsulamiento básico en la Práctica 5, la gestión de paquetes y despliegue modular en la Práctica 6, la jerarquía profunda de herencia entre cuadriláteros en la Práctica 7, hasta el polimorfismo dinámico mediante interfaces gráficas y operativas en la Práctica 8. El problema técnico y académico de esta entrega consiste en unificar y estandarizar formalmente todos los artefactos de modelado empleando el Lenguaje Unificado de Modelado (UML), generando diagramas de clases individuales y globales, diagramas de paquetes, diagramas de actividades y secuencias reproducibles nativamente en Draw.io e integrándolos coherentemente en este informe técnico.

1.2. Motivación

La documentación visual y estructurada constituye una fase crítica en la ingeniería de software moderna, pues proporciona un puente inequívoco entre los requerimientos analíticos y las construcciones sintácticas en Java. Analizar retrospectivamente las cuatro prácticas precedentes bajo el prisma de UML permite consolidar los principios fundamentales de diseño, verificar la cohesión y el acoplamiento de las clases, transparentar las decisiones de multiplicidad y justificar el uso de técnicas como la agregación, la herencia profunda y la realización de contratos abstractos. Asimismo, la elaboración de diagramas nativos y editables asegura la trazabilidad, mantenibilidad y verificación rigurosa del software ante auditorías académicas y profesionales.

1.3. Objetivos

El objetivo principal de la presente práctica es modelar, estandarizar e integrar la totalidad de los diagramas UML estructurales y de comportamiento correspondientes a las Prácticas 5, 6, 7 y 8, utilizando la herramienta Draw.io conforme a los estándares internacionales de la OMG. Como objetivos específicos se establecen: (1) construir diagramas de clases individuales completos con tipado, visibilidad estricta y miembros declarados; (2) elaborar diagramas estructurales globales que expliciten dependencias, asociaciones, composiciones y generalizaciones con sus respectivas cardinalidades demostrables; (3) trazar diagramas de flujo y secuencias temporales para operaciones y escenarios de ejecución representativos; y (4) redactar un reporte técnico integral con preámbulo formal y paginación editorial cuidada que analice la evolución de la arquitectura orientada a objetos del equipo.

2. Marco Teórico

2.1. Fundamentos del Lenguaje Unificado de Modelado (UML)

El Lenguaje Unificado de Modelado (UML) es el estándar industrial adoptado por el *Object Management Group* (OMG) para la especificación, visualización, construcción y documentación de los artefactos que componen un sistema de software orientado a objetos [1]. Surgido a mediados de la década de 1990 a partir de la unificación de los métodos de Grady Booch, James Rumbaugh y Ivar Jacobson, UML ofrece una notación rica y rigurosa que modela tanto la dimensión estática (estructural) como la dimensión dinámica (comportamiento e interacción) de las aplicaciones informáticas [2]. En el contexto pedagógico de la programación en Java, UML permite abstraer los detalles sintácticos del compilador para concentrarse en las relaciones esenciales entre componentes, garantizando que el diseño guíe a la implementación y no a la inversa.

2.2. Diagramas Estructurales: Clases y Paquetes

Los diagramas estructurales representan la configuración estática del sistema, capturando las entidades conceptuales, sus partes internas y los enlaces permanentes que las articulan [5]:

- **Diagrama de Clases:** Modela el vocabulario del sistema mediante rectángulos divididos en tres compartimentos horizontales: nombre de la clase (pudiendo incluir estereotipos como «interface» o nombres abstractos en cursiva), lista de atributos tipados con visibilidad, y lista de operaciones o métodos con parámetros y tipos de retorno.
- **Diagrama de Paquetes:** Proporciona un mecanismo de agrupamiento de alto nivel para organizar elementos del modelo en módulos o espacios de nombres jerárquicos, representados por carpetas con pestaña. Permite controlar la visibilidad a nivel de biblioteca y gestionar dependencias entre subsistemas compilados en archivos JAR.

2.3. Anatomía de una Clase y Convenciones Notacionales

En UML, cada compartimento de la clase sigue una sintaxis normalizada que traduce fielmente las construcciones del lenguaje Java [3]:

- **Visibilidad:** Se señalan con símbolos unificados: público (`+`), privado (`-`), protegido (`#`) y de paquete o por defecto (`~`).

- **Atributos:** Siguen el formato `visibilidad nombre: Tipo [multiplicidad] = valorInicial`. Los miembros de clase (estáticos) se distinguen por estar subrayados, y las constantes inmutables por la propiedad `{readOnly}`.
- **Operaciones:** Se especifican como `visibilidad nombre(param1: Tipo, ...): TipoRetorno`. Los constructores carecen de tipo de retorno, y las operaciones abstractas se escriben en tipografía cursiva.

2.4. Relaciones entre Clases y Cardinalidades

Las relaciones expresan cómo colaboran o se vinculan las entidades dentro del modelo conceptual:

1. **Asociación:** Conexión estructural semántica entre clases, trazada con una línea sólida continua. Puede ser bidireccional o dirigida, y se acompaña de multiplicidades en los extremos (e.g., `1`, `0..1`, `1..*`, `*`).
2. **Agregación:** Relación del tipo "todo-parte" no destructiva, donde los componentes pueden subsistir independientemente del contenedor. Se representa con un rombo blanco o vacío en el extremo del contenedor.
3. **Composición:** Forma fuerte de agregación con dependencia existencial estricta; las partes nacen y fenecen junto con el objeto continente. Se denota mediante un rombo relleno o sólido.
4. **Generalización (Herencia):** Relación taxonómica donde una subclase hereda atributos y métodos de una superclase común, posibilitando la especialización y reutilización de código. Se ilustra con una línea continua y una flecha triangular cerrada y vacía que apunta a la superclase.
5. **Realización (Implementación):** Vínculo contractual entre una interfaz y la clase que suministra la lógica de sus métodos abstractos. Se traza con una línea discontinua o punteada rematada en una flecha triangular vacía.
6. **Dependencia:** Relación transitoria de uso donde una clase requiere de otra temporalmente (por ejemplo, como parámetro o variable local en un método). Se representa mediante una flecha discontinua abierta.

2.5. Diagramas de Comportamiento: Secuencia y Actividades

Para modelar los aspectos dinámicos de las soluciones implementadas se emplean dos diagramas fundamentales [4]:

- **Diagrama de Secuencia:** Destaca la ordenación cronológica de los mensajes intercambiados entre líneas de vida que representan objetos participantes. Los

mensajes sincrónicos usan flechas continuas con punta sólida, mientras que los retornos opcionales se grafican con flechas discontinuas abiertas.

- **Diagrama de Actividades (Flujo):** Describe el flujo de control lógico, decisiones y transformaciones en algoritmos críticos. Emplea rectángulos redondeados para inicio/fin, rombos para decisiones binarias (Sí/No) y rectángulos para pasos de procesamiento.

El modelado dinámico resulta indispensable para complementar la visión estática de clases, garantizando que el paso de parámetros, la instanciación temporal y el despacho polimórfico de métodos se verifiquen visualmente antes de la ejecución física del código en la máquina virtual Java.

3. Desarrollo

3.1. Modelado UML de la Práctica 5: Encapsulamiento y Control

La Práctica 5 sentó las bases del encapsulamiento formal en el lenguaje Java mediante dos casos de estudio fundamentales: el modelado de una entidad laboral (`Empleado`) y el registro temporal de fechas de calendario (`Fecha`), controlados respectivamente por clases conductoras con métodos principales deterministas.

3.1.1. Clase Empleado

La clase `Empleado` encapsula tres atributos privados: `nombre`, `apellido` y `salarioMensual`. Su constructor inicializa los valores y delega la asignación del sueldo al método mutador `setSalarioMensual`, el cual aplica una regla de negocio estricta: el salario debe ser estrictamente positivo (> 0), conservando el valor previo o descartando valores no conformes. La figura 3.1 presenta la estructura estática de la clase.

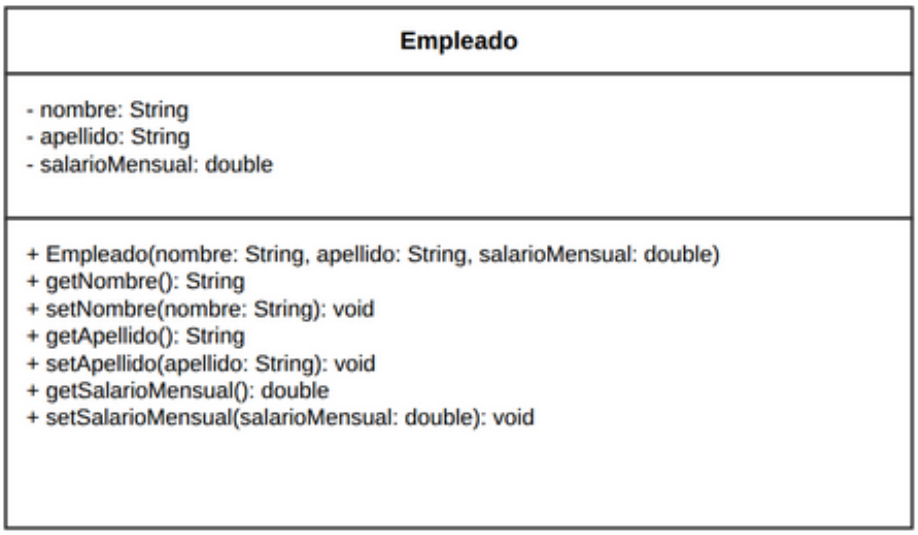


Figura 3.1: Diagrama UML formal de la clase Empleado (Práctica 5).

El flujo operativo correspondiente a la validación y actualización salarial en `setSalarioMensual` se ilustra en la figura 3.2, evidenciando la bifurcación condicional ante montos válidos e inválidos.

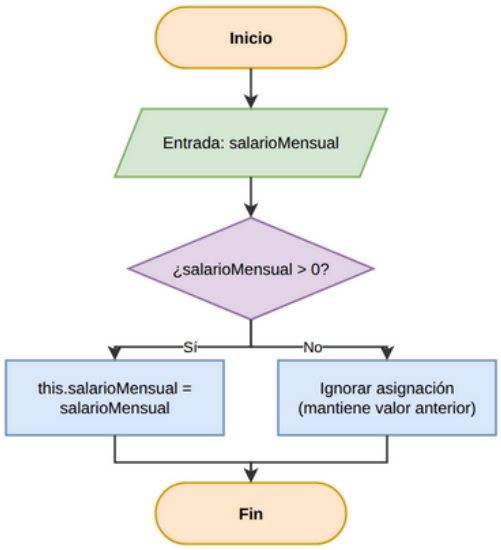


Figura 3.2: Diagrama de actividades del método setSalarioMensual en Empleado.

3.1.2. Clase Fecha

La clase `Fecha` modela una terna numérica entera conformada por `mes`, `dia` y `anio`. Proporciona métodos de actualización individual y la operación `mostrarFecha` para

emitir el formato convencional mes/día/año por la consola estándar. Su diseño UML se exhibe en la figura 3.3.

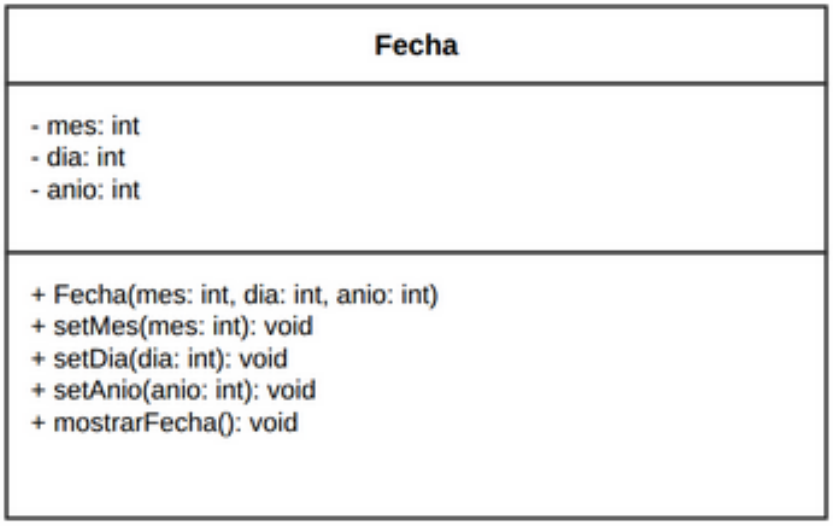


Figura 3.3: Diagrama UML formal de la clase Fecha (Práctica 5).

3.1.3. Clases Conductoras y Diagrama Global de Práctica 5

Las clases `MainEmpleado` y `MainFecha` actúan como puntos de entrada (`main`) para la interacción secuencial con el usuario. Las figuras 3.4 y 3.5 muestran sus diagramas individuales.

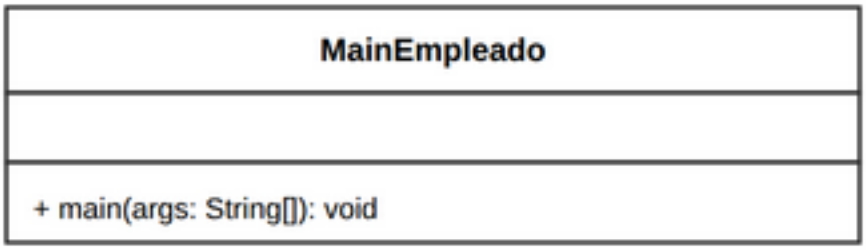


Figura 3.4: Diagrama UML de la clase conductora MainEmpleado.

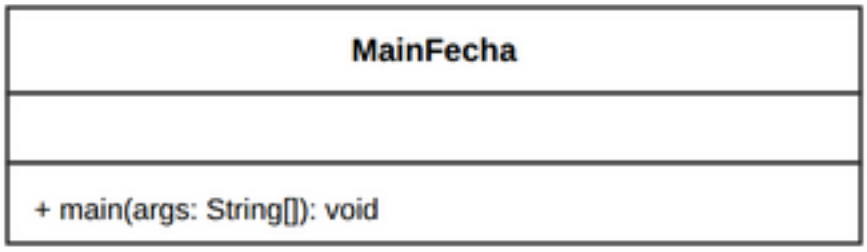


Figura 3.5: Diagrama UML de la clase conductora MainFecha.

El diagrama global de la Práctica 5 (figura 3.6) articula las cuatro clases, demostrando que `MainEmpleado` instancia y utiliza dos objetos de tipo `Empleado` (multiplicidad 1 a 2), mientras que `MainFecha` instancia un único objeto `Fecha` (multiplicidad 1 a 1), ambas mediante relaciones de dependencia («instancia»).

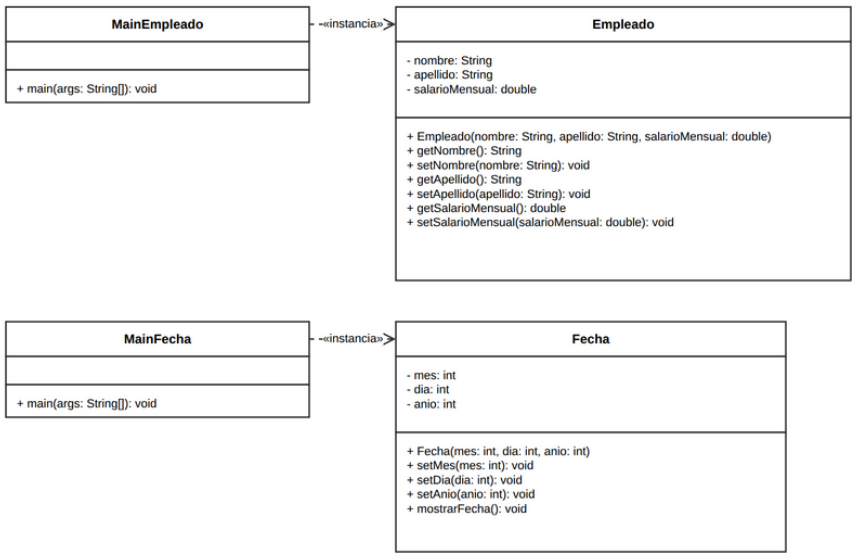


Figura 3.6: Diagrama UML global de integración de la Práctica 5.

Para evidenciar la dinámica temporal de cálculo salarial y el otorgamiento del diez por ciento de aumento anual, la figura 3.7 documenta el diagrama de secuencia entre el usuario, la clase controladora y las dos instancias creadas.

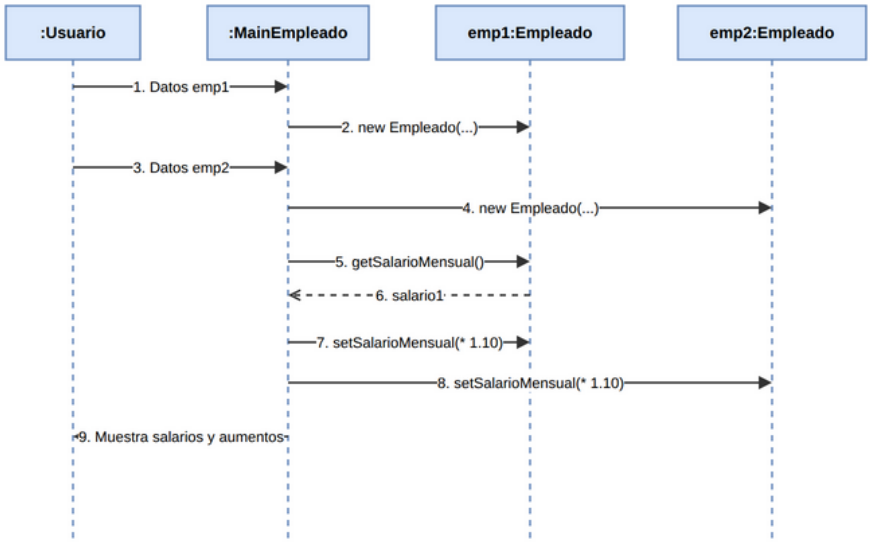


Figura 3.7: Diagrama de secuencia de interacción salarial en MainEmpleado.

3.2. Modelado UML de la Práctica 6: Organización Modular y Paquetes

La Práctica 6 marcó un hito metodológico al recopilar ejercicios previos y proyectos en una estructura formalizada de paquetes bajo el prefijo institucional `mx.unam.fi.die.poo.g7`, compilando todas las clases y dependencias hacia un artefacto ejecutable distribuible `Practica6.jar`.

3.2.1. Diagrama de Paquetes y Arquitectura Modular

El esquema modular separa claramente las responsabilidades del sistema: cálculo básico en `p1`, estructuras de control en `p2`, procesamiento de cadenas en `p3`, análisis de oraciones en `p4`, entidades laborales en `p5` y el sistema de gestión hospitalaria en `proyecto`. La figura 3.8 modela la arquitectura de paquetes convergente al archivo JAR.

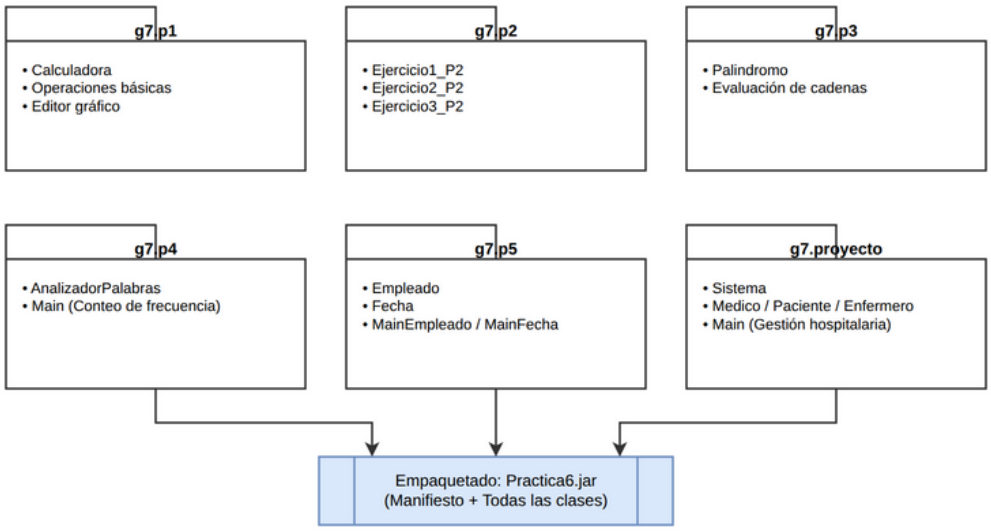


Figura 3.8: Diagrama de paquetes y empaquetado JAR de la Práctica 6.

3.2.2. Analizador de Palabras y Colecciones

En el paquete `p4`, la clase `AnalizadorPalabras` explota la estructura asociativa `Map<String, Integer>` para registrar frecuencias léxicas en una oración. Su estructura UML se presenta en la figura 3.9, mientras que el algoritmo de tokenización e inserción de frecuencias se ilustra en el diagrama de actividades de la figura 3.10.

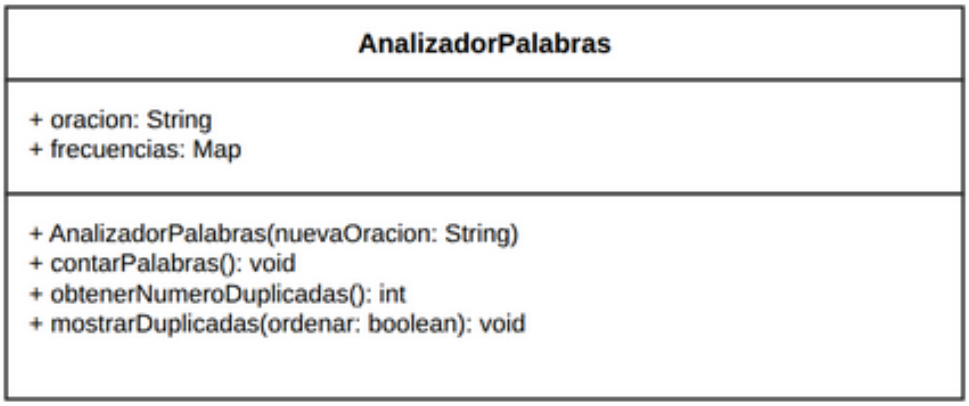


Figura 3.9: Diagrama UML de la clase AnalizadorPalabras (paquete p4).

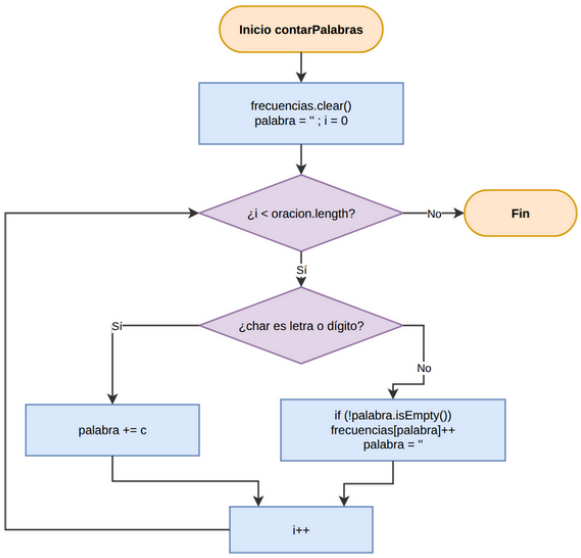


Figura 3.10: Diagrama de flujo del método contarPalabras en AnalizadorPalabras.

3.2.3. Sistema Hospitalario

El subsistema clínico desarrollado en el paquete `proyecto` vincula la clase central `Sistema` con el personal sanitario (`Medico`, `Enfermero`) y los pacientes (`Paciente`). La relación estructural es de agregación directa: el hospital contiene listas dinámicas (`ArrayList`) de profesionales y enfermos, pero su ciclo vital conceptual no destruye físicamente los registros. Las figuras 3.11 y 3.12 detallan el modelo del sistema y su mapa consolidado.

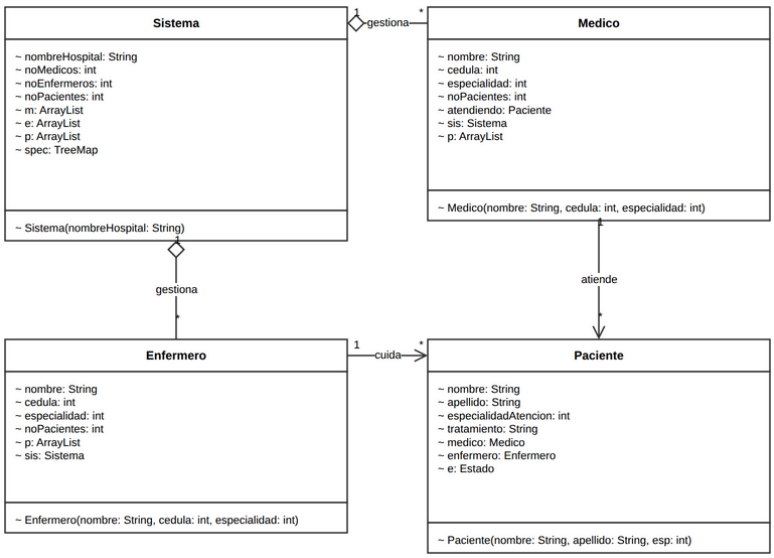


Figura 3.11: Diagrama UML de clases del Sistema Hospitalario.

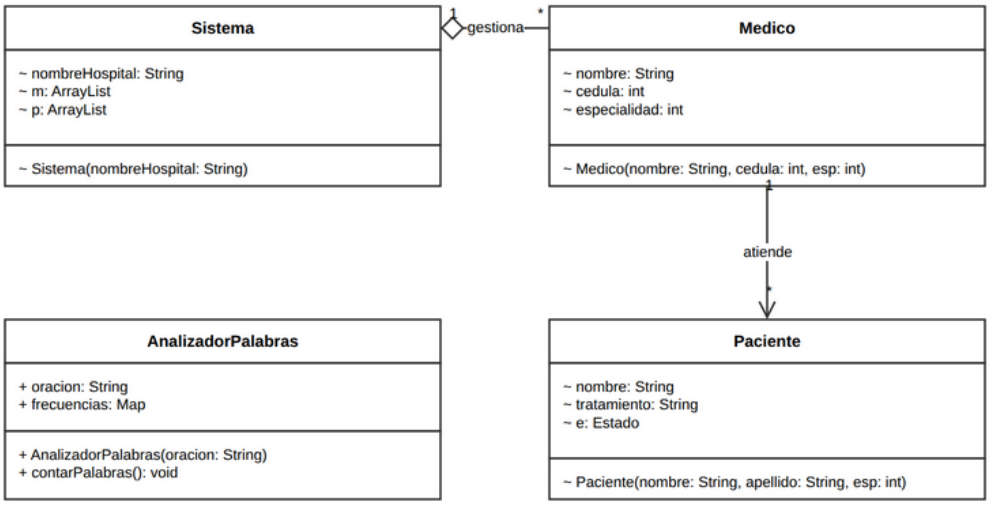


Figura 3.12: Diagrama UML global de integración de la Práctica 6.

La interacción dinámica de registro clínico, alta de médicos y asignación de consultas se modela en el diagrama de secuencia de la figura 3.13.

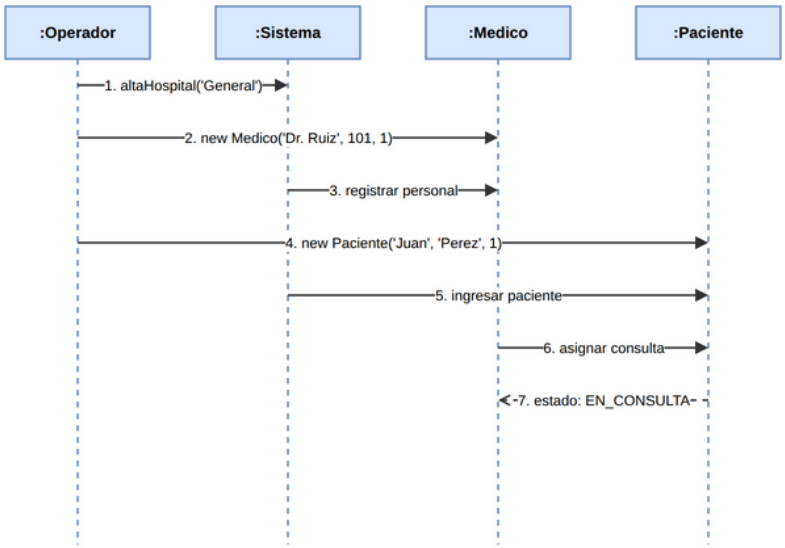


Figura 3.13: Diagrama de secuencia del flujo de registro y consulta hospitalaria.

3.3. Modelado UML de la Práctica 7: Jerarquía Profunda de Cuadriláteros

La Práctica 7 abordó el paradigma de la herencia profunda al concebir un árbol taxonómico de cinco niveles fundado en la geometría analítica bidimensional, apoyándose en una clase base de coordenadas `Punto`.

3.3.1. Composición de Coordenadas y Superclase Cuadrilatero

La clase `Punto` modela el par coordenado (x, y) con constructores sobrecargados y métodos de acceso, como se aprecia en la figura 3.14. La superclase común `Cuadrilatero` (figura 3.15) almacena cuatro vértices `p1`, `p2`, `p3` y `p4`. La relación entre ambas es de estricta composición: un cuadrilátero está formado intrínsecamente por cuatro vértices indivisibles.

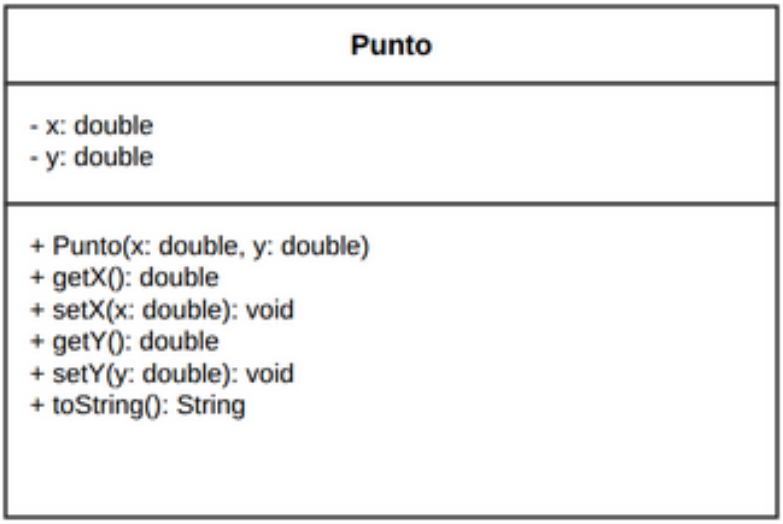


Figura 3.14: Diagrama UML de la clase Punto (Práctica 7).

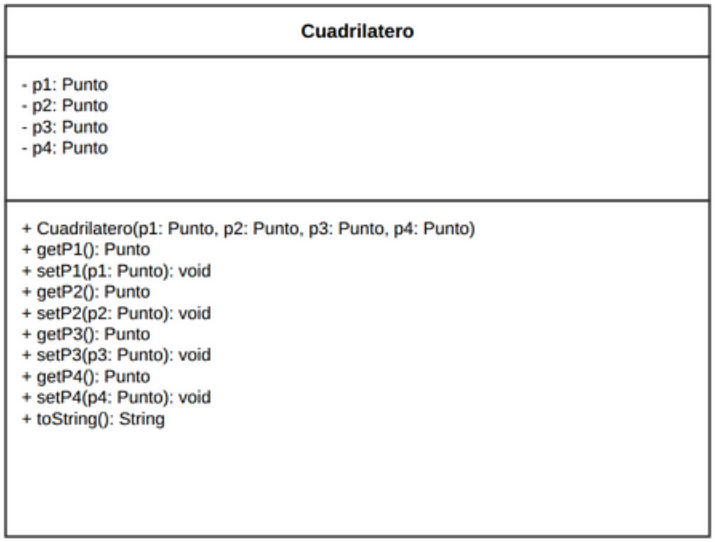


Figura 3.15: Diagrama UML de la superclase Cuadrilatero.

3.3.2. Niveles de Especialización Geométrica

La jerarquía profundiza de forma progresiva refinando el cálculo de superficie:

1. `Trapezoide` (figura 3.16): Hereda de `Cuadrilatero` e introduce fórmulas para base mayor, base menor y altura horizontal, implementando Área $= \frac{(B+b)\cdot h}{2}$.
2. `Paralelogramo` (figura 3.17): Especializa a `Trapezoide` al forzar lados opuestos iguales ($B = b$), reduciendo el cálculo a Área $= b \cdot h$.
3. `Rectangulo` (figura 3.18): Especializa a `Paralelogramo` asumiendo ángulos rectos de noventa grados (Área = longitud $\cdot$ ancho).
4. `Cuadrado` (figura 3.19): Caso límite de `Rectangulo` con lados congruentes (Área = lado^2).

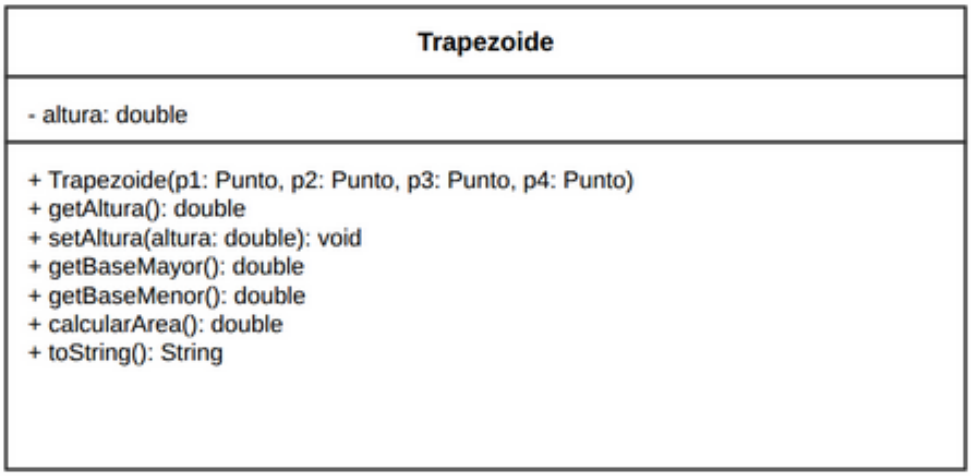


Figura 3.16: Diagrama UML de la clase Trapezoide.

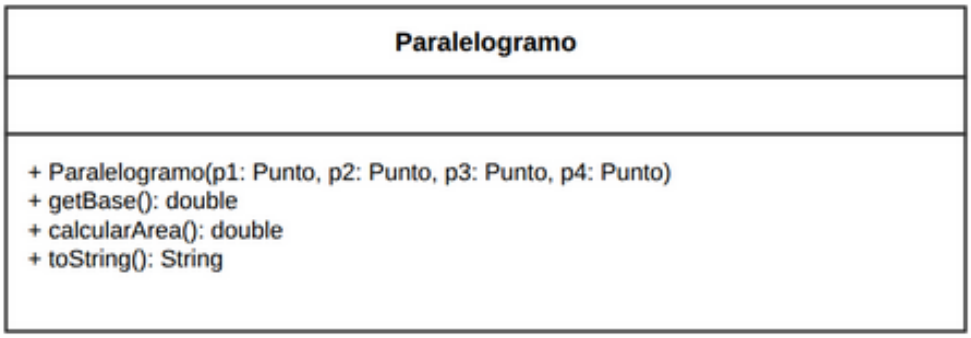


Figura 3.17: Diagrama UML de la clase Paralelogramo.

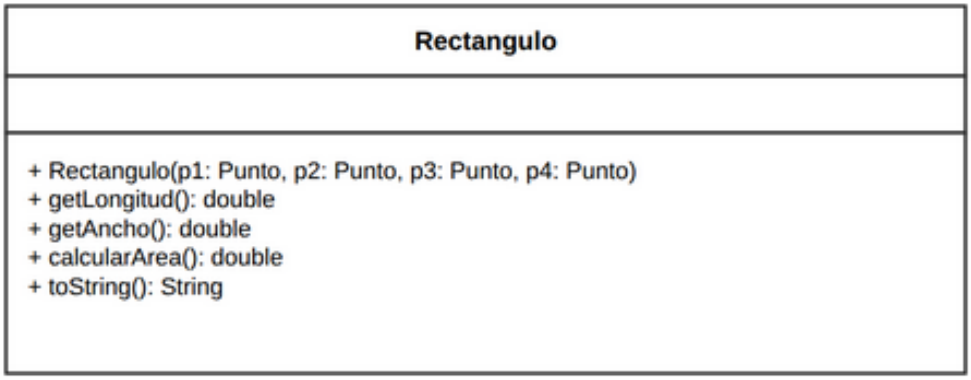


Figura 3.18: Diagrama UML de la clase Rectangulo.

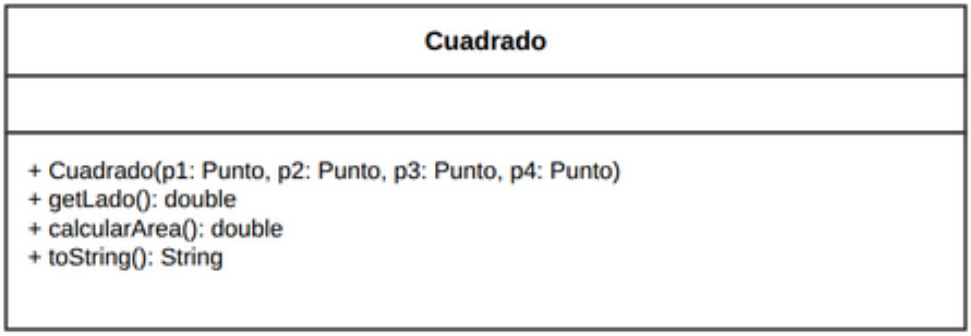


Figura 3.19: Diagrama UML de la clase Cuadrado.

La clase conductora `Main` (figura 3.20) se encarga de crear las figuras geométricas con coordenadas representativas y reportar sus dimensiones.

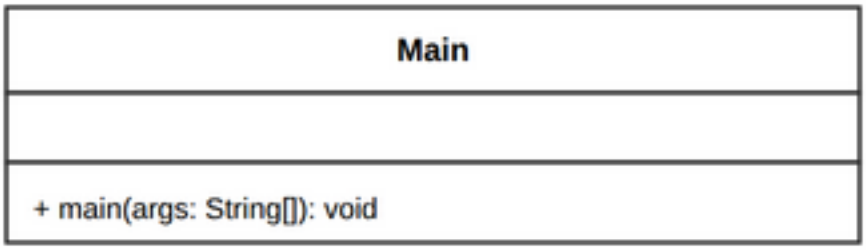


Figura 3.20: Diagrama UML de la clase conductora Main (Práctica 7).

3.3.3. Jerarquía Global, Flujo y Secuencia Polimórfica

La figura 3.21 integra la cadena genealógica completa y su composición con `Punto`. La figura 3.22 esquematiza el flujo lógico de evaluación de áreas, mientras que la figura 3.23 traza la invocación secuencial de `calcularArea()` con resolución dinámica.

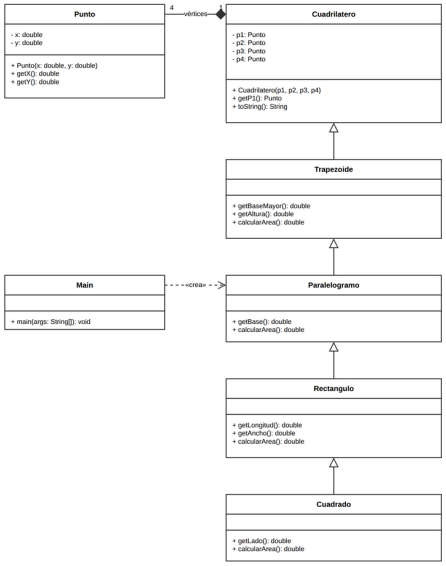


Figura 3.21: Jerarquía profunda global de herencia en cuadriláteros.

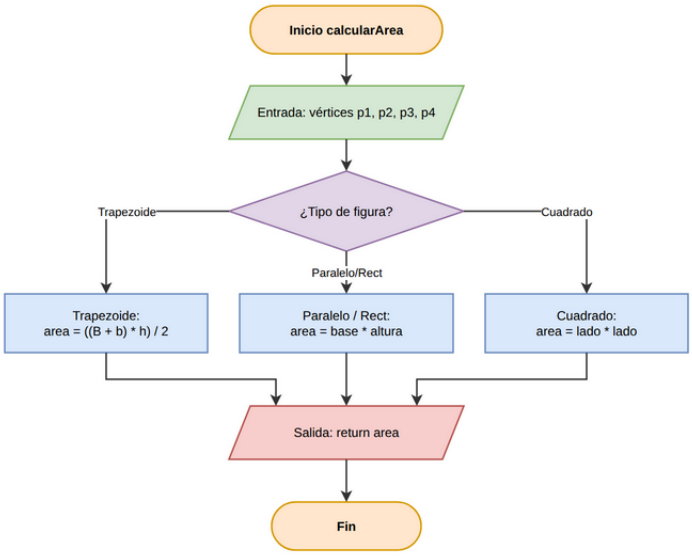


Figura 3.22: Diagrama de flujo algorítmico del cálculo de áreas geométricas.

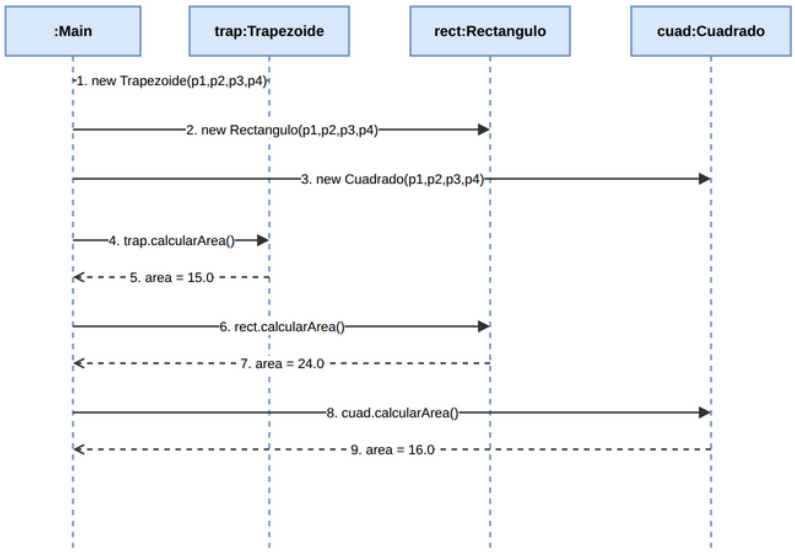


Figura 3.23: Diagrama de secuencia de invocación dinámica en cuadriláteros.

3.4. Modelado UML de la Práctica 8: Interfaces y Polimorfismo

La Práctica 8 representó la culminación de la abstracción desacoplada mediante el uso de la interfaz `HuellaCarbono`, implementada por tres clases conceptualmente disjuntas: `Edificio`, `Auto` y `Bicicleta`, complementada con una ventana gráfica de inicio de sesión (`LoginGUI`).

3.4.1. Contrato de Interfaz y Clases Implementadoras

En la figura 3.24 se presenta la interfaz `HuellaCarbono`, la cual declara la operación abstracta `getHuellaCarbono()` con tipo de retorno `double`. Las tres clases realizan este contrato conforme a sus propios modelos de emisión educativa expresados en kilogramos de dióxido de carbono equivalente (kg CO_2e):

- `Edificio` (figura 3.25): Multiplica el consumo eléctrico anual en kWh por el factor de emisión eléctrica.
- `Auto` (figura 3.26): Estima la emisión a partir del kilometraje anual, rendimiento en litros cada cien kilómetros y factor de gasolina.
- `Bicicleta` (figura 3.27): Registra un factor de emisión operativa directo igual a cero (0.0).

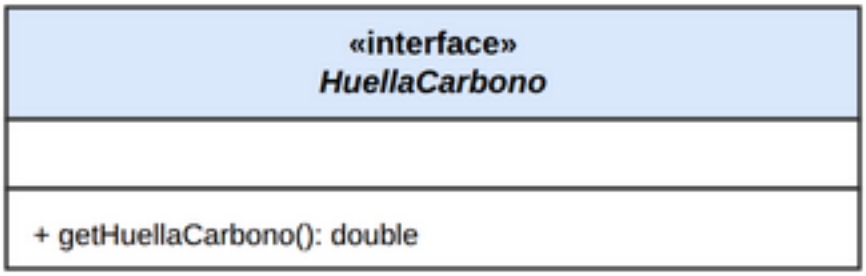


Figura 3.24: Diagrama UML de la interfaz HuellaCarbono.

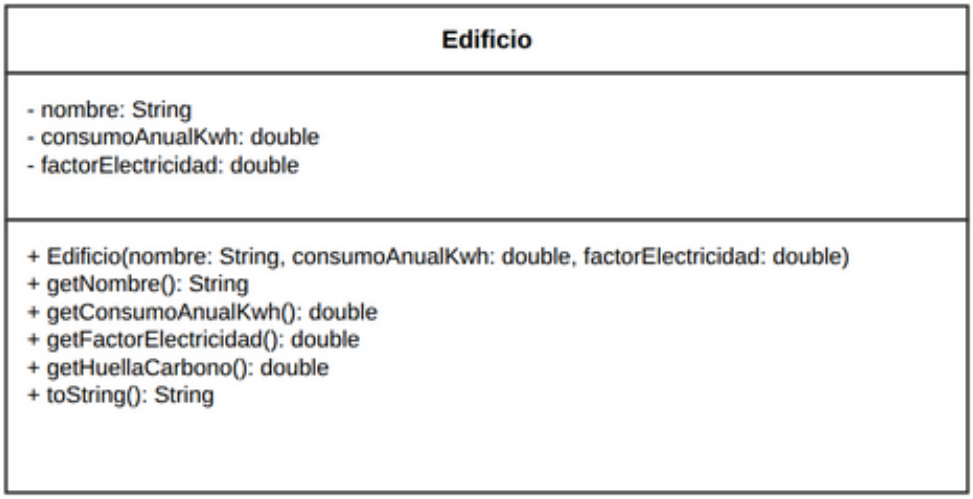


Figura 3.25: Diagrama UML de la clase Edificio.

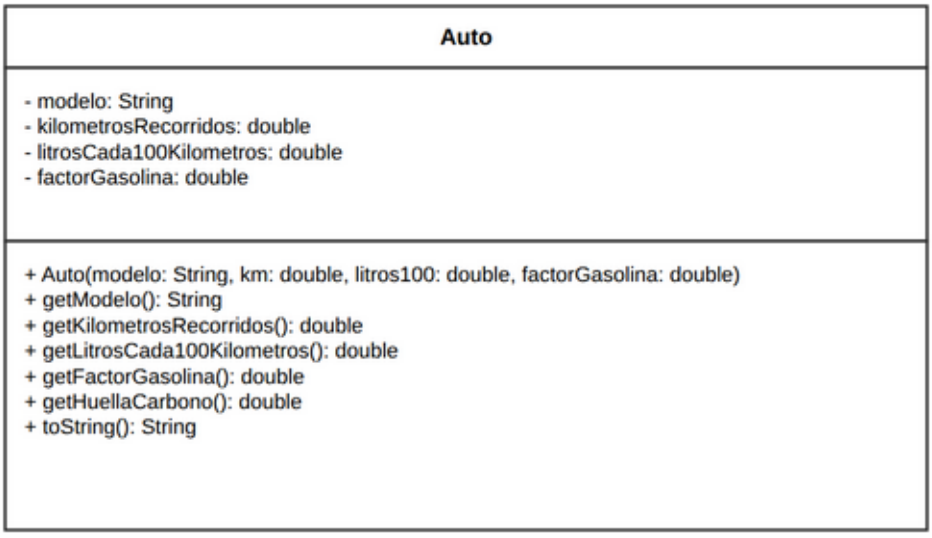


Figura 3.26: Diagrama UML de la clase Auto.

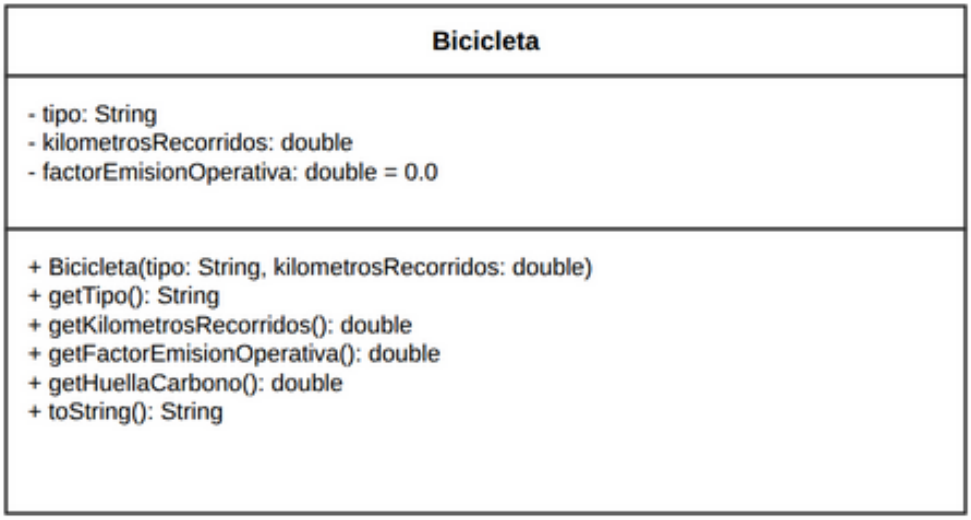


Figura 3.27: Diagrama UML de la clase Bicicleta.

3.4.2. Interfaz Gráfica y Clase Principal

La clase `LoginGUI` (figura 3.28) extiende de `JFrame` e implementa `ActionListener` para gestionar eventos gráficos con campos protegidos de usuario y contraseña. Por su parte, la clase `Main` (figura 3.29) inicializa el contenedor polimórfico.

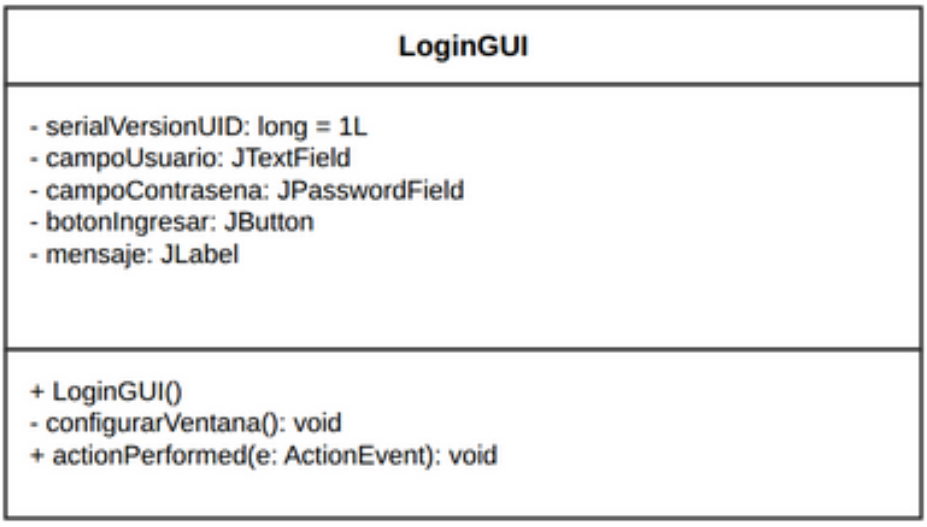


Figura 3.28: Diagrama UML de la interfaz gráfica LoginGUI.

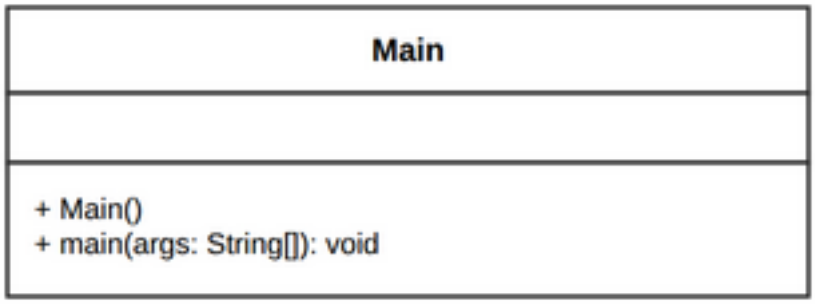


Figura 3.29: Diagrama UML de la clase principal Main (Práctica 8).

3.4.3. Integración Global, Flujo Polimórfico y Secuencia

La figura 3.30 documenta la realización de la interfaz mediante líneas discontinuas con punta triangular y la agregación en la lista polimórfica `ArrayList<HuellaCarbono>`. El flujo del ciclo de cálculo e impresión se presenta en la figura 3.31, y la secuencia de despacho dinámico de mensajes en la figura 3.32.

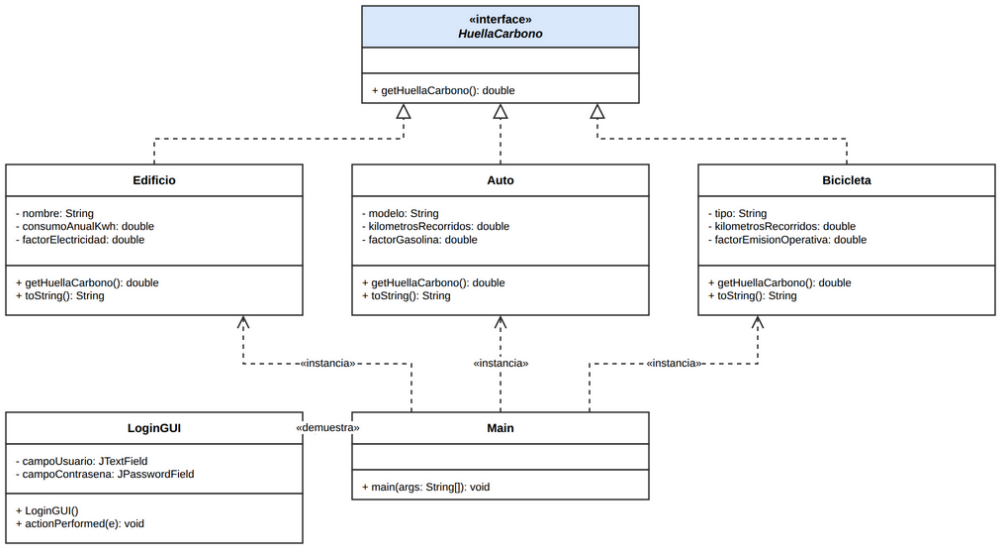


Figura 3.30: Diagrama UML global de polimorfismo con interfaces.

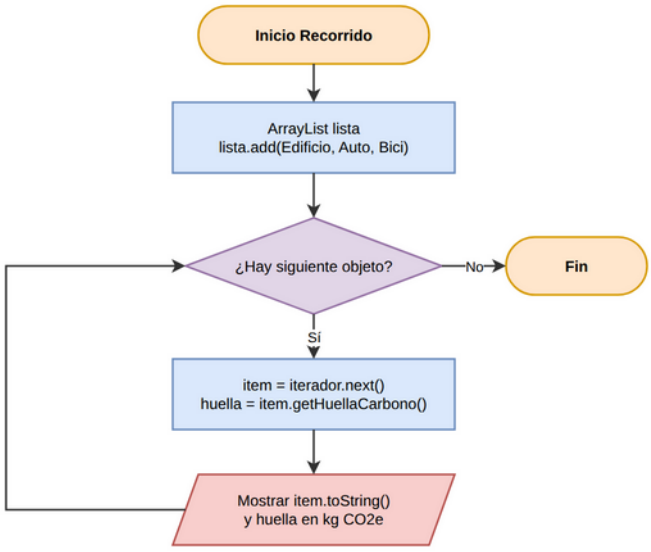


Figura 3.31: Diagrama de actividades del recorrido polimórfico de huellas.

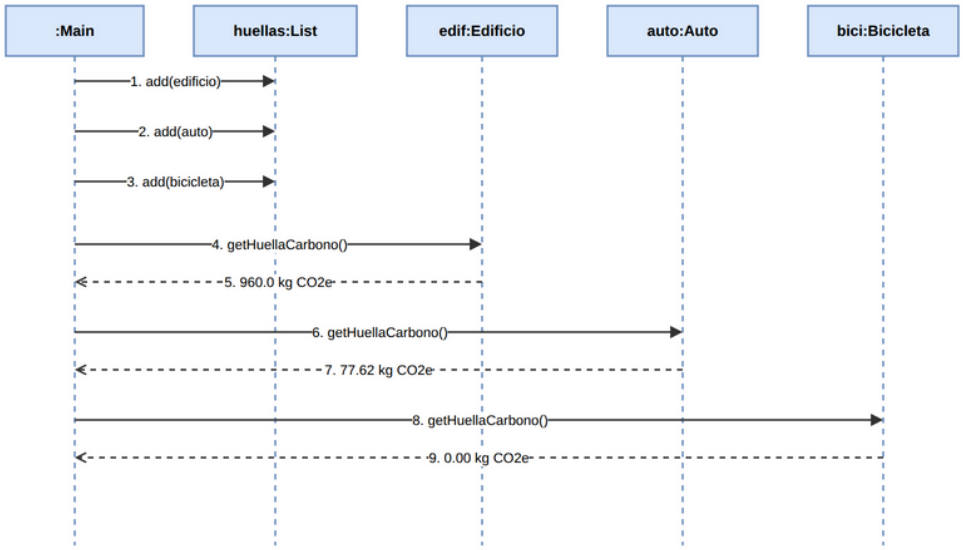


Figura 3.32: Diagrama de secuencia de la invocación polimórfica en tiempo de ejecución.

3.5. Metamodelo Comparativo y Visión Global de Práctica 9

Como síntesis arquitectónica, la figura 3.33 establece un metamodelo evolutivo que articula los cuatro paradigmas abordados en las prácticas 5 a 8, demostrando cómo el modelado UML unifica la encapsulación, empaquetado modular, herencia profunda y realización de interfaces en un cuerpo conceptual sólido y coherente.

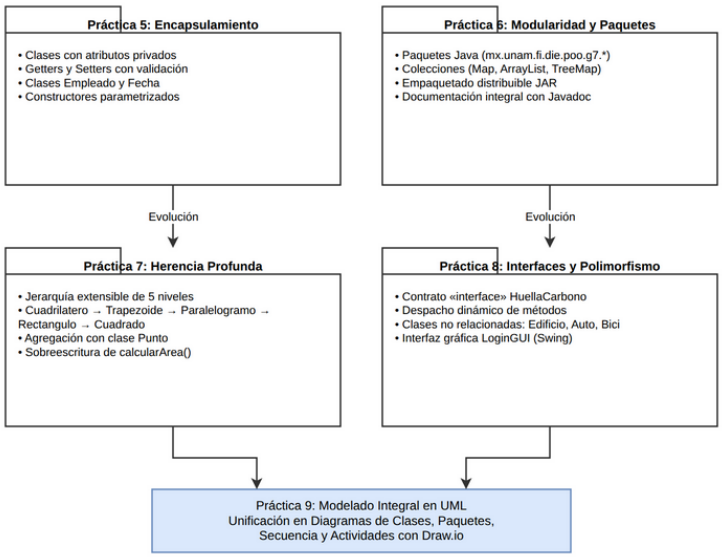


Figura 3.33: Metamodelo arquitectónico comparativo de las Prácticas 5 a 8 en UML.

4. Resultados

4.1. Verificación de Conformidad Estructural y Sintáctica

La totalidad de los 33 diagramas generados en Draw.io fue contrastada de manera exhaustiva contra las fuentes compiladas en Java de los repositorios de las prácticas 5, 6, 7 y 8. La tabla 1 resume la cobertura de artefactos gráficos entregados en `doc/Diagramas/`.

Práctica	**Concepto Clave**	**Archivos .drawio**	**Renders .png**
Práctica 5	Encapsulamiento, Clases Empleado/Fecha y Conductoras	7	7
Práctica 6	Modularidad, Paquetes JAR y Sistema Hospitalario	6	6
Práctica 7	Jerarquía Profunda de Herencia en Cuadriláteros	10	10
Práctica 8	Polimorfismo con Interfaces y Ventana Gráfica	9	9
Práctica 9	Metamodelo Comparativo e Integración Arquitectónica	1	1
Total	**Ecosistema completo de modelado UML**	**33**	**33**

Tabla 1: Inventario y categorización de los 33 diagramas UML entregados.

4.2. Análisis de Trazabilidad entre Modelos y Código

La inspección de los diagramas confirma los siguientes hallazgos metodológicos:

1. **Precisión de Miembros y Visibilidad:** Se conservó con fidelidad la visibilidad estricta: los atributos de `Empleado`, `Fecha`, `Punto`, `Cuadrilatero`, `Edificio`, `Auto` y `Bicicleta` se modelaron como privados (`-`), mientras que en el sistema hospitalario se representaron con visibilidad de paquete (`~`), reflejando con exactitud las declaraciones originales de código.

2. **Distinción Rigurosa entre Herencia e Interfaces:** En la Práctica 7 se utilizó exclusivamente la generalización con línea sólida (`Cuadrilatero` ← `Trapezoide` ← `Paralelogramo` ← `Rectangulo` ← `Cuadrado`), mientras que en la Práctica 8 se aplicó la realización con línea discontinua («interface» `HuellaCarbono` ··· ◀ implementaciones), evitando confusiones conceptuales.

3. **Legibilidad Editorial:** Todas las figuras integradas en el documento respetan un tamaño tipográfico mínimo equivalente a 10 puntos en la impresión carta, evitando deformaciones de escala, solapamiento de conectores o cruces innecesarios de flechas.

4. **Acoplamiento y Encapsulamiento Modular (Práctica 6):** El diagrama de paquetes ilustra la separación de responsabilidades entre subsistemas independientes (`p1` a `p5`) y el núcleo hospitalario (`proyecto`), demostrando cómo el uso del modificador de paquete (`~`) protege el estado interno de las entidades clínicas frente a manipulaciones externas, mientras que la compilación hacia un único archivo JAR garantiza la portabilidad y cohesión arquitectónica del despliegue.

5. **Jerarquía Taxonómica y Sustitución de Liskov (Práctica 7):** La cadena profunda de herencia entre cuadriláteros respeta estrictamente el principio de sustitución de Liskov, pues cualquier instancia derivada de `Cuadrado`, `Rectangulo` o `Paralelogramo` puede ser manipulada como un `Cuadrilatero` sin alterar la integridad de sus coordenadas constituyentes (`Punto`), permitiendo que el método `calcularArea()` se refine de forma polimórfica en cada nivel taxonómico.

6. **Desacoplamiento Contractual e Inversión de Dependencias (Práctica 8):** La interfaz `HuellaCarbono` materializa la inversión de dependencias y segregación contractual, permitiendo que la clase conductora gestione colecciones polimórficas de entidades heterogéneas (`Edificio`, `Auto`, `Bicicleta`) invocando dinámicamente `getHuellaCarbono()` en tiempo de ejecución, completamente desacoplada de la interfaz gráfica interactiva `LoginGUI`.

5. Conclusiones

La realización de la Práctica 9 ha demostrado de manera fehaciente que el Lenguaje Unificado de Modelado no es un mero accesorio documental posterior a la programación, sino una disciplina de abstracción indispensable para concebir, refinar y comunicar la arquitectura interna de sistemas orientados a objetos. Al modelar retrospectivamente los códigos de las prácticas 5 a 8, fue posible constatar visualmente la progresión cualitativa del diseño: desde el aislamiento básico de estado mediante encapsulamiento estricto, pasando por la delimitación modular de espacios de nombres en paquetes, hasta la sofisticación de jerarquías profundas de especialización y contratos polimórficos independientes de la implementación.

Asimismo, la experiencia práctica con Draw.io y la estandarización de la notación de la OMG puso de manifiesto la relevancia de seleccionar la simbología exacta para cada tipo de acoplamiento. Diferenciar con precisión entre una composición existencial de vértices en cuadriláteros, una agregación dinámica de personal en un hospital y una realización de interfaz para el cálculo de huellas ecológicas transforma diagramas ambiguos en auténticas especificaciones ejecutables de ingeniería.

Finalmente, la integración automatizada de artefactos gráficos nativos y su renderización vectorial hacia reportes en LaTeX garantiza la repetibilidad, trazabilidad y legibilidad profesional requeridas en la formación tecnológica universitaria, sentando bases metodológicas firmes para el desarrollo y presentación de proyectos de mayor escala.

Referencias

[1] Booch, G., Rumbaugh, J., Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2da ed.). Addison-Wesley Professional.

[2] Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3ra ed.). Addison-Wesley Professional.

[3] Deitel, P., Deitel, H. (2017). *Java How to Program, Early Objects* (11ma ed.). Pearson Educación.

[4] Gamma, E., Helm, R., Johnson, R., Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

[5] Sommerville, I. (2011). *Software Engineering* (9na ed.). Addison-Wesley.